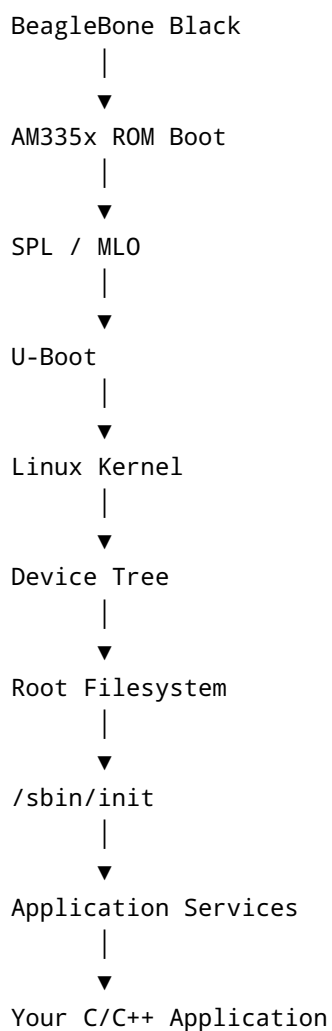


BeagleBone Black BSP and Complete Linux OS From Scratch

1. Objective

This document explains how to create a complete Linux BSP and bootable operating-system image for the **BeagleBone Black (BBB)** from scratch, without using Yocto, Buildroot, OpenEmbedded, or another automated embedded-Linux distribution builder.

The final result will contain:



The BSP will contain:

```
BSP
├─ Cross Compiler
├─ Bootloader
│   └─ SPL
│       └─ U-Boot
├─ Linux Kernel
├─ Device Tree
├─ Root Filesystem
├─ Libraries
├─ Kernel Modules
├─ Firmware
├─ Init System
├─ Application
└─ Flashable SD/eMMC Image
```

The final board should be able to:

1. Power on.
2. Execute the AM335x ROM bootloader.
3. Load SPL.
4. Load U-Boot.
5. Initialize DDR and peripherals.
6. Load Linux kernel.
7. Load the BeagleBone Black device tree.
8. Mount the root filesystem.
9. Start `/sbin/init`.
10. Initialize `/dev`, `/proc`, `/sys`, `/run`.
11. Start networking and other required services.
12. Start your C/C++ application automatically.

2. Hardware Used

The target is:

BeagleBone Black

Main SoC:

```
Texas Instruments AM335x
ARM Cortex-A8
```

ARMv7-A
32-bit

Typical BBB memory/storage:

RAM:
512 MB DDR3

eMMC:
4 GB

Boot media:
microSD
eMMC

Typical peripherals:

UART
I2C
SPI
GPIO
PWM
ADC
USB
Ethernet
MMC/eMMC

For the initial BSP development, it is strongly recommended to use a **microSD card** instead of immediately modifying the eMMC.

3. Host PC

Use a Linux development machine.

For example:

Ubuntu Linux
x86_64

Install only normal development utilities required to build the individual components.

Example:

```
sudo apt update

sudo apt install
  build-essential
  git
  wget
  curl
  bc
  bison
  flex
  libssl-dev
  libelf-dev
  libncurses-dev
  device-tree-compiler
  rsync
  cpio
  python3
  python3-pip
  u-boot-tools
  dosfstools
  e2fsprogs
  parted
  fdisk
  xz-utils
  bzip2
  gzip
  tar
  gcc
  g++
```

We are **not** installing:

```
Yocto
OpenEmbedded
BitBake
Buildroot
PTXdist
OpenWrt build system
```

Every major BSP component will be built and assembled manually.

4. Create BSP Workspace

Create one directory for the entire BSP.

```
mkdir -p ~/bbb-bsp  
cd ~/bbb-bsp
```

Create the following structure:

```
mkdir -p  
  sources  
  toolchain  
  sysroot  
  build  
  images  
  rootfs  
  deploy  
  scripts  
  downloads
```

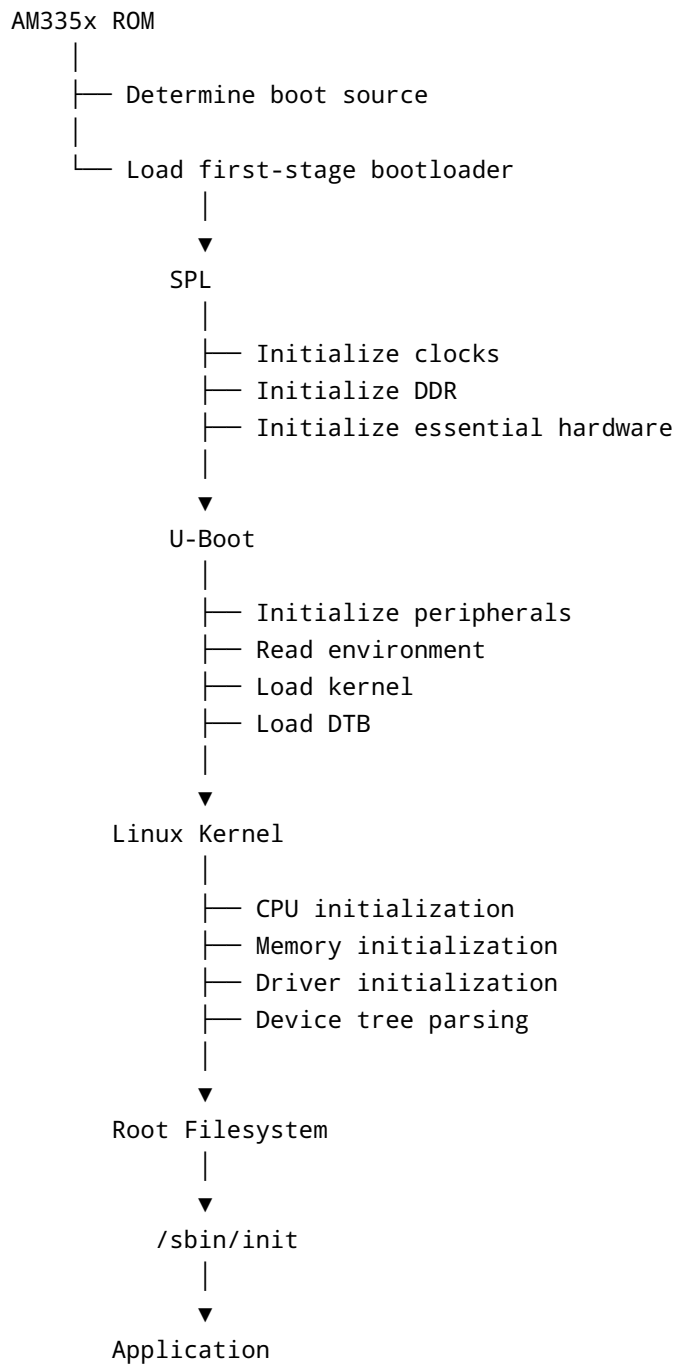
Final workspace:

```
bbb-bsp/  
├─ sources/  
├─ toolchain/  
├─ sysroot/  
├─ build/  
├─ images/  
├─ rootfs/  
├─ deploy/  
├─ scripts/  
└─ downloads/
```

5. Understand the Complete Boot Process

Before building anything, understand the BBB boot sequence.

When power is applied:



This distinction is important:

BSP

The BSP is the board-specific software required to make Linux run on the board.

It includes:

```
Bootloader
Kernel
Device Tree
Drivers
Firmware
Toolchain/sysroot
Board configuration
```

Root filesystem

The rootfs contains user-space software:

```
/bin
/sbin
/etc
/lib
/usr
/var
/home
/dev
/proc
/sys
/run
```

Application

Your application is above the BSP:

```
BSP
|
├─ Kernel
├─ Drivers
├─ Libraries
└─ Rootfs
    |
    └─ Your Application
```

6. Select the Cross Compiler

The BBB uses ARMv7-A.

The target architecture is:

```
ARM 32-bit  
ARMv7  
Cortex-A8  
hard-float  
EABI
```

A practical compiler prefix is:

```
arm-linux-gnueabihf-
```

Therefore:

```
arm-linux-gnueabihf-gcc  
arm-linux-gnueabihf-g++  
arm-linux-gnueabihf-ld  
arm-linux-gnueabihf-ar  
arm-linux-gnueabihf-objcopy
```

For the first BSP implementation, you can use an existing GNU cross compiler installed on the host.

For example:

```
sudo apt install gcc-arm-linux-gnueabihf  
g++-arm-linux-gnueabihf
```

Verify:

```
arm-linux-gnueabihf-gcc --version
```

And:

```
arm-linux-gnueabihf-gcc -dumpmachine
```

Expected:

```
arm-linux-gnueabihf
```


Set environment variables:

```
export ARCH=arm
export CROSS_COMPILE=arm-linux-gnueabihf-
```

Add these to your shell configuration later if required.

7. Build a Proper Toolchain From Scratch

If the objective is truly to understand BSP development from the lowest level, eventually build the toolchain yourself.

The toolchain consists approximately of:

```
Binutils
  ↓
Linux kernel headers
  ↓
GCC stage 1
  ↓
glibc
  ↓
GCC final
  ↓
libgcc
  ↓
libstdc++
```

The resulting environment is:

```
arm-linux-gnueabihf-gcc
  |
  ▼
  glibc
  |
  ├── libc
  ├── libm
  ├── librt
  ├── libpthread
  └── dynamic loader
```

For production BSP work, maintain a dedicated sysroot:

```
bbb-bsp/sysroot/  
├─ usr/  
├─ lib/  
├─ include/  
└─ etc/
```

The sysroot is extremely important because your C/C++ application must compile against the same user-space ABI that exists on the target board.

8. Build Binutils

Download a GNU Binutils source release.

Extract it:

```
cd ~/bbb-bsp/sources  
  
tar xf binutils-*.tar.xz  
mkdir -p ~/bbb-bsp/build/binutils  
cd ~/bbb-bsp/build/binutils
```

Configure:

```
../../sources/binutils-*/configure  
--target=arm-linux-gnueabi  
--prefix=$HOME/bbb-bsp/toolchain  
--with-sysroot=$HOME/bbb-bsp/sysroot
```

Build:

```
make -j$(nproc)
```

Install:

```
make install
```

Verify:

```
~/bbb-bsp/toolchain/bin/arm-linux-gnueabi-hf-ld --version
```

9. Linux Kernel Headers

The C library requires Linux kernel headers.

Download Linux kernel source.

Extract:

```
cd ~/bbb-bsp/sources
tar xf linux-*.tar.xz
cd linux-*
```

Set:

```
export ARCH=arm
export CROSS_COMPILE=arm-linux-gnueabi-hf-
```

Install headers:

```
make mrproper
make ARCH=arm CROSS_COMPILE=$CROSS_COMPILE multi_v7_defconfig
make ARCH=arm CROSS_COMPILE=$CROSS_COMPILE
INSTALL_HDR_PATH=$HOME/bbb-bsp/sysroot/usr
headers_install
```

Now:

```
sysroot/usr/include/
```

contains the kernel UAPI headers required by user-space software.

10. Build GCC Stage 1

GCC requires prerequisites.

Inside the GCC source tree:

```
./contrib/download_prerequisites
```

Configure a minimal compiler:

```
mkdir -p ~/bbb-bsp/build/gcc-stage1  
cd ~/bbb-bsp/build/gcc-stage1
```

Then configure GCC for the target.

The initial compiler is used to bootstrap the target environment.

Important GCC options include:

```
--target=arm-linux-gnueabi  
--prefix=...  
--with-sysroot=...  
--enable-languages=c
```

Build:

```
make -j$(nproc) all-gcc  
make install-gcc
```

At this stage:

```
C compiler  
↓  
can generate ARM binaries
```

but it is not yet a complete Linux C/C++ development environment.

11. Build glibc

glibc provides the main Linux C runtime.

It supplies:

```
libc
libm
libpthread
librt
dynamic loader
etc.
```

Configure glibc against the target sysroot.

The target must use the correct ARM floating-point ABI.

For BBB:

```
ARM hard-float
```

Build and install:

```
make -j$(nproc)
make install
```

The exact glibc configuration depends on the selected GCC/binutils versions, so keep the binutils, GCC and glibc versions compatible.

After installation, the sysroot should contain things similar to:

```
sysroot/
├─ lib/
│   ├── ld-linux-armhf.so.3
│   ├── libc.so.6
│   └─ ...
├─ usr/
│   ├── include/
│   └─ lib/
└─ ...
```

12. Build Final GCC

Now build the complete compiler.

Enable:

```
C  
C++
```

and required runtime libraries.

Typical target:

```
arm-linux-gnueabihf
```

Build:

```
make -j$(nproc)
```

Install:

```
make install
```

Verify:

```
arm-linux-gnueabihf-gcc --version  
arm-linux-gnueabihf-g++ --version
```

Test:

```
cat > hello.c <<'EOF'  
#include <stdio.h>  
  
int main(void)  
{  
    printf("Hello BeagleBone Black\n");  
    return 0;  
}  
EOF
```

Compile:

```
arm-linux-gnueabihf-gcc hello.c -o hello
```

Check:

```
file hello
```

You should see an ARM executable.

13. Build U-Boot

U-Boot is the second-stage bootloader.

Create source directory:

```
cd ~/bbb-bsp/sources
```

Get U-Boot source and extract it.

Set:

```
export ARCH=arm  
export CROSS_COMPILE=arm-linux-gnueabihf-
```

Enter U-Boot:

```
cd u-boot-*
```

Clean:

```
make mrproper
```

Configure BeagleBone Black:

```
make am335x_evm_defconfig
```

Build:

```
make -j$(nproc)
```

Important output files include:

```
MLO / SPL
u-boot.img
```

Depending on the U-Boot version/configuration, the exact generated filenames may vary.

Check:

```
ls -lh MLO
ls -lh u-boot.img
```

The bootloader should be built specifically for the AM335x/BeagleBone platform.

14. Understand SPL and U-Boot

The BBB bootloader has two important stages.

SPL

SPL means:

```
Secondary Program Loader
```

Its job is to perform enough initialization to load the full U-Boot.

Typical responsibilities:

```
CPU initialization
Clock initialization
DDR initialization
Basic pin configuration
MMC initialization
Load U-Boot
```

Then:

```
SPL
↓
U-Boot
```


U-Boot

U-Boot performs more complete initialization and provides a boot environment.

It can:

```
Read SD/eMMC
Read FAT
Read ext4
Load kernel
Load DTB
Set boot arguments
Configure network
Start Linux
```

15. Build the Linux Kernel

Download Linux kernel source.

```
cd ~/bbb-bsp/sources
```

Extract:

```
tar xf linux-*.tar.xz
cd linux-*
```

Set:

```
export ARCH=arm
export CROSS_COMPILE=arm-linux-gnueabihf-
```

Clean:

```
make mrproper
```

For BeagleBone, begin with a known-good ARM multi-platform configuration:

```
make multi_v7_defconfig
```

Then customize:

```
make menuconfig
```

16. Kernel Configuration

Important kernel options include:

CPU

```
ARM architecture
AM33xx support
Cortex-A8
```

Device Tree

Enable:

```
Device Tree support
```

MMC

Enable:

```
MMC support
SDHCI
OMAP HSMMC
```

This is necessary to access SD/eMMC.

EXT4

Enable:

```
EXT4 filesystem
```

Preferably built into the kernel:

```
CONFIG_EXT4_FS=y
```

rather than:

```
CONFIG_EXT4_FS=m
```

because the root filesystem may be ext4 and must be available before modules can be loaded.

USB

Enable required USB host/device support.

Ethernet

Enable:

```
TI CPSW Ethernet
```

for the AM335x Ethernet controller.

Serial Console

Enable the TI OMAP serial driver.

For example:

```
CONFIG_SERIAL_8250
CONFIG_SERIAL_8250_CONSOLE
```

depending on the selected kernel configuration/version.

GPIO

Enable GPIO support.

I2C

Enable:

```
I2C
OMAP I2C
```

SPI

Enable:

```
SPI
OMAP SPI
```

PWM

Enable the required PWM subsystem.

17. Build Kernel

Build:

```
make -j$(nproc) zImage dtbs
```

The kernel will generate:

```
arch/arm/boot/zImage
```

Device-tree files will be under:

```
arch/arm/boot/dts/
```

Depending on the kernel release, the BBB DTB will have a name similar to:

```
am335x-boneblack.dtb
```

Verify:

```
ls -lh arch/arm/boot/zImage
```

and:

```
find arch/arm/boot/dts -name '*boneblack*.dtb'
```

18. Kernel Modules

If kernel modules are required:

```
make -j$(nproc) modules
```

Install them into your root filesystem:

```
make modules_install  
INSTALL_MOD_PATH=$HOME/bbb-bsp/rootfs
```

You should get:

```
rootfs/lib/modules/<kernel-version>/
```

19. What Is Device Tree?

The Linux kernel needs information about the board hardware.

Instead of hard-coding every board detail in the kernel, Linux uses a Device Tree.

The BBB Device Tree describes:

```
CPU  
RAM  
UART  
I2C  
SPI  
MMC  
Ethernet  
GPIO  
USB  
PWM  
LED
```

Expansion headers
Pin multiplexing

For example conceptually:

```
AM335x
├── UART
├── I2C
├── SPI
├── MMC
├── Ethernet
├── GPIO
└── USB
```

Device Tree:

```
.dts
  ↓
.dtb
  ↓
U-Boot
  ↓
Linux kernel
```

20. Modify Device Tree

Never modify the generated `.dtb` directly.

Modify the source:

```
am335x-boneblack.dts
```

and any included `.dtsi` files.

For example, to enable a peripheral:

```
Device Tree source
|
▼
```

```
Configure peripheral
|
▼
Compile with dtc
|
▼
.dtb
```

Compile manually if needed:

```
dtc
-I dts
-O dtb
-o am335x-boneblack.dtb
am335x-boneblack.dts
```

Normally the Linux kernel build system should compile the DTB for you.

21. Build BusyBox

Now create the user-space environment.

BusyBox provides small versions of common Linux utilities:

```
sh
ls
cp
mv
mkdir
mount
umount
cat
echo
ps
dmesg
ip
ifconfig
grep
kill
```

Download BusyBox source.

Extract:

```
cd ~/bbb-bsp/sources
tar xf busybox-*.tar.xz
cd busybox-*
```

Set:

```
export ARCH=arm
export CROSS_COMPILE=arm-linux-gnueabi-
```

Configure:

```
make defconfig
```

For the first system, make BusyBox statically linked.

```
make menuconfig
```

Enable:

```
Busybox Settings
  Build Options
    Build BusyBox as a static binary
```

Build:

```
make -j$(nproc)
```

Install:

```
make CONFIG_PREFIX=$HOME/bbb-bsp/rootfs install
```

Now:

```
rootfs/bin/busybox
rootfs/bin/sh
rootfs/bin/ls
```



```
rootfs/bin/mount
...
```

exist.

22. Create Root Filesystem Directory Structure

Create:

```
cd ~/bbb-bsp/rootfs

mkdir -p
  bin
  sbin
  lib
  lib/modules
  etc
  proc
  sys
  dev
  run
  tmp
  root
  home
  usr/bin
  usr/sbin
  usr/lib
  var
  var/log
  var/run
  mnt
  opt
```

The minimum rootfs structure is:

```
/
├─ bin
├─ sbin
├─ dev
├─ etc
├─ home
├─ lib
└─ mnt
```

```
|— opt
|— proc
|— root
|— run
|— sys
|— tmp
|— usr
└— var
```

23. Create /etc/passwd

Create:

```
nano rootfs/etc/passwd
```

Example:

```
root:x:0:0:root:/root:/bin/sh
```

24. Create /etc/group

```
nano rootfs/etc/group
```

Example:

```
root:x:0:
```

25. Create /etc/hostname

```
echo "beaglebone" > rootfs/etc/hostname
```

26. Create /etc/inittab

BusyBox init can use:

```
/etc/inittab
```

Example:

```
::sysinit:/etc/init.d/rcS
ttyS0::askfirst:-/bin/sh
::restart:/sbin/init
::ctrlaltdel:/sbin/reboot
```

The exact serial device must match the UART used by your BBB configuration.

27. Create System Initialization Script

Create:

```
mkdir -p rootfs/etc/init.d
nano rootfs/etc/init.d/rcS
```

Example:

```
#!/bin/sh

echo "=====
echo " BeagleBone Black Linux Startup "
echo "=====

mount -t proc proc /proc
mount -t sysfs sysfs /sys
mount -t devtmpfs devtmpfs /dev

mkdir -p /dev/pts
mount -t devpts devpts /dev/pts

mkdir -p /run
mkdir -p /tmp
```

```
hostname beaglebone

echo "Root filesystem initialized"

# Start application
if [ -x /usr/bin/myapp ]; then
    /usr/bin/myapp &
fi
```

Make executable:

```
chmod +x rootfs/etc/init.d/rcS
```

28. /dev Handling

There are several ways to manage `/dev`.

For a modern Linux system, use:

```
devtmpfs
```

The kernel automatically creates device nodes.

Therefore:

```
mount -t devtmpfs devtmpfs /dev
```

is normally sufficient.

For early bring-up, you can also create critical device nodes manually.

For example:

```
sudo mknod rootfs/dev/console c 5 1
sudo mknod rootfs/dev/null c 1 3
```

However, do not manually create every device node if devtmpfs is being used.

29. /proc

Linux process and kernel information is exposed through:

```
/proc
```

Mount:

```
mount -t proc proc /proc
```

You can then use:

```
cat /proc/cpuinfo  
cat /proc/meminfo  
cat /proc/mounts
```

30. /sys

Kernel device information is exposed through:

```
/sys
```

Mount:

```
mount -t sysfs sysfs /sys
```

This is important for:

```
GPIO  
devices  
drivers  
LEDs  
USB  
MMC  
network devices
```

31. /dev/pts

For terminal and shell operation:

```
mkdir -p /dev/pts  
mount -t devpts devpts /dev/pts
```

32. Dynamic C/C++ Applications

A statically linked BusyBox is convenient for initial bring-up.

However, a normal C/C++ application may require:

```
libc  
libgcc  
libstdc++  
libm  
dynamic loader  
other shared libraries
```

For example:

```
arm-linux-gnueabi-g++ main.cpp -o myapp
```

Check:

```
file myapp
```

and:

```
arm-linux-gnueabi-readelf -d myapp
```

You may see:

```
NEEDED  
    libstdc++.so.6
```

```
libgcc_s.so.1  
libc.so.6
```

The target rootfs must contain those libraries.

33. Copy Required C/C++ Libraries

From the target sysroot, populate:

```
rootfs/lib/  
rootfs/usr/lib/
```

Typical libraries include:

```
libc.so.6  
libm.so.6  
libpthread.so.0  
libdl.so.2  
libgcc_s.so.1  
libstdc++.so.6  
ld-linux-armhf.so.3
```

Do not randomly copy libraries from an x86 PC.

They must be ARM libraries compatible with the target ABI.

34. Create a C Application

Create:

```
mkdir -p ~/bbb-bsp/application  
cd ~/bbb-bsp/application
```

Create:

```
nano main.c
```

Example:

```

#include <stdio.h>
#include <unistd.h>

int main(void)
{
    printf("=====\n");
    printf(" BeagleBone Black Application\n");
    printf("=====\n");

    while (1)
    {
        printf("Application is running...\n");
        sleep(5);
    }

    return 0;
}

```

Compile:

```

arm-linux-gnueabi-gcc
main.c
-o myapp

```

35. C++ Application

Create:

```

#include <iostream>
#include <unistd.h>

int main()
{
    std::cout << "BeagleBone Black C++ Application"
              << std::endl;

    while (true)
    {
        std::cout << "Application is running..."
                  << std::endl;

        sleep(5);
    }
}

```



```
}  
  
    return 0;  
}
```

Compile:

```
arm-linux-gnueabi-g++  
    main.cpp  
    -o myapp
```

36. Install Application Into Rootfs

```
cp myapp ~/bbb-bsp/rootfs/usr/bin/myapp
```

Set executable:

```
chmod +x ~/bbb-bsp/rootfs/usr/bin/myapp
```

Now:

```
rootfs/  
├─ usr/  
│   └─ bin/  
│       └─ myapp
```

37. Add Application to Startup

The earlier:

```
/etc/init.d/rcS
```

can start:

```
/usr/bin/myapp &
```

For example:

```
if [ -x /usr/bin/myapp ]; then
    echo "Starting application..."
    /usr/bin/myapp &
fi
```

Then the boot flow becomes:

```
Kernel
↓
init
↓
rcS
↓
myapp
```

38. Create Network Configuration

For a simple Ethernet setup:

```
mkdir -p rootfs/etc/network
```

Create:

```
rootfs/etc/network/interfaces
```

Example:

```
auto lo
iface lo inet loopback

auto eth0
iface eth0 inet dhcp
```

If using BusyBox, the startup script can run:

```
ip link set eth0 up  
udhcpc -i eth0
```

or use the networking system included in your rootfs.

39. Create Basic /etc/resolv.conf

For DNS testing:

```
echo "nameserver 8.8.8.8" > rootfs/etc/resolv.conf
```

For production, configure the DNS according to your network.

40. Create Root Filesystem Image

At first, it is easier to create the filesystem directly on an SD-card partition.

You can also create an ext4 image.

Example:

```
dd if=/dev/zero  
  of=images/rootfs.ext4  
  bs=1M  
  count=1024
```

Create filesystem:

```
mkfs.ext4 images/rootfs.ext4
```

Mount:

```
sudo mkdir -p /mnt/bbb-rootfs  
sudo mount -o loop images/rootfs.ext4 /mnt/bbb-rootfs
```

Copy:

```
sudo cp -a rootfs/. /mnt/bbb-rootfs/
```

Sync:

```
sync
```

Unmount:

```
sudo umount /mnt/bbb-rootfs
```

41. Prepare Boot Partition

The boot partition should contain the boot files.

Example:

```
boot/  
├─ MLO  
├─ u-boot.img  
├─ zImage  
├─ am335x-boneblack.dtb  
└─ extlinux/  
    └─ extlinux.conf
```

Copy:

```
mkdir -p images/boot  
  
cp <u-boot-build>/MLO  
  images/boot/  
  
cp <u-boot-build>/u-boot.img  
  images/boot/  
  
cp <kernel>/arch/arm/boot/zImage  
  images/boot/
```

```
cp <kernel>/arch/arm/boot/dts/.../am335x-boneblack.dtb
images/boot/
```

The exact DTB path depends on the Linux kernel version.

42. U-Boot Boot Configuration

A convenient way to configure modern U-Boot is:

```
extlinux/extlinux.conf
```

Create:

```
mkdir -p images/boot/extlinux
```

Create:

```
images/boot/extlinux/extlinux.conf
```

Example:

```
TIMEOUT 3
DEFAULT linux

LABEL linux
    KERNEL /zImage
    FDT /am335x-boneblack.dtb
    APPEND console=tty00,115200n8 root=/dev/mmcblk0p2 rw rootwait
```

The console name can differ depending on the kernel version/device-tree configuration. If your kernel uses `ttyS0`, use that instead.

The important concept is:

```
console=<serial-device>,115200
```

and:

```
root=<root filesystem partition>
```

43. Kernel Command Line

The kernel command line tells Linux where the root filesystem exists.

Example:

```
root=/dev/mmcblk0p2  
rootwait  
rw  
console=tty00,115200n8
```

Meaning:

```
root=/dev/mmcblk0p2
```

Use second SD-card partition as root filesystem.

```
rootwait
```

Wait for MMC storage to become available.

```
rw
```

Mount root filesystem read/write.

```
console=...
```

Send kernel console output to UART.

44. SD Card Partition Layout

Use two partitions.

Recommended:

```
SD Card
|
├── Partition 1
│   ├── FAT32
│   └── Boot
└── Partition 2
    ├── ext4
    └── Root filesystem
```

Example:

```
/dev/sdX
|
├── /dev/sdX1  FAT32
└── /dev/sdX2  EXT4
```

Do not accidentally use your PC's system disk.

Verify the device carefully:

```
lsblk
```

45. Create SD Card Partition Table

Insert the SD card.

Identify it:

```
lsblk
```

Suppose it is:

```
/dev/sdb
```

Unmount existing partitions:

```
sudo umount /dev/sdb1 2>/dev/null  
sudo umount /dev/sdb2 2>/dev/null
```

Partition:

```
sudo fdisk /dev/sdb
```

Create:

```
Partition 1:  
  FAT32  
  ~128 MB  
  
Partition 2:  
  Linux  
  Remaining space
```

For example:

```
/dev/sdb1  
/dev/sdb2
```

46. Format Partitions

Format boot:

```
sudo mkfs.vfat -F 32 /dev/sdb1
```

Format rootfs:

```
sudo mkfs.ext4 /dev/sdb2
```

47. Mount SD Card

```
sudo mkdir -p /mnt/bbb-boot  
sudo mkdir -p /mnt/bbb-root
```

Mount:

```
sudo mount /dev/sdb1 /mnt/bbb-boot  
sudo mount /dev/sdb2 /mnt/bbb-root
```

48. Install Boot Files

Copy:

```
sudo cp images/boot/ML0  
    /mnt/bbb-boot/  
  
sudo cp images/boot/u-boot.img  
    /mnt/bbb-boot/  
  
sudo cp images/boot/zImage  
    /mnt/bbb-boot/  
  
sudo cp images/boot/am335x-boneblack.dtb  
    /mnt/bbb-boot/
```

Create:

```
sudo mkdir -p /mnt/bbb-boot/extlinux
```

Copy configuration:

```
sudo cp images/boot/extlinux/extlinux.conf  
    /mnt/bbb-boot/extlinux/
```

49. Install Root Filesystem

Copy:

```
sudo cp -a rootfs/. /mnt/bbb-root/
```

If kernel modules were built:

```
sudo cp -a  
  rootfs/lib/modules  
  /mnt/bbb-root/lib/
```

Fix ownership if necessary:

```
sudo chown -R root:root /mnt/bbb-root
```

50. Sync SD Card

Very important:

```
sync
```

Unmount:

```
sudo umount /mnt/bbb-boot  
sudo umount /mnt/bbb-root
```

Then:

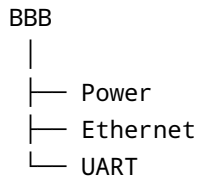
```
sync
```

Remove the SD card safely.

51. Insert SD Card Into BeagleBone Black

Insert the SD card into the BBB.

Connect:



For first bring-up, a UART serial console is highly recommended.

Use a **3.3 V TTL USB-UART adapter** suitable for the BBB UART pins.

Do not connect a normal RS-232 voltage-level interface directly to the BBB UART.

52. Serial Console

Typical serial configuration:

```
Baud rate:    115200
Data bits:    8
Parity:       None
Stop bits:    1
Flow control: None
```

Example using Linux:

```
screen /dev/ttyUSB0 115200
```

or:

```
minicom -D /dev/ttyUSB0 -b 115200
```

53. Boot the Board

Power the BBB.

The expected output is conceptually:

```
U-Boot SPL ...  
DRAM: ...  
Loading U-Boot...
```

Then:

```
U-Boot ...  
CPU: AM335x  
DRAM: 512 MiB  
MMC: ...
```

Then:

```
Loading kernel...  
Loading device tree...  
Starting kernel...
```

Then Linux:

```
Booting Linux...
```

Eventually:

```
BeagleBone Black Linux Startup
```

and:

```
Application is running...
```

54. First Debugging Point: ROM → SPL

If absolutely nothing appears:

Check:

```
Power
SD card
SD card partition
MLO
boot configuration
UART
```

The first boot stage is extremely sensitive.

Check that:

```
MLO
```

is present in the boot partition and that the SD card is correctly formatted.

55. Second Debugging Point: SPL → U-Boot

If SPL starts but U-Boot does not:

Check:

```
u-boot.img
DDR initialization
MMC access
U-Boot configuration
```

The serial log is critical.

56. Third Debugging Point: U-Boot → Kernel

If you reach:

```
U-Boot #
```

but Linux does not start:

Check:

```
zImage  
DTB  
boot command  
kernel address  
DTB address  
bootargs
```

At the U-Boot prompt:

```
printenv
```

You can inspect:

```
bootcmd  
bootargs  
fdtfile  
kernel_addr_r  
fdt_addr_r
```

57. Manually Boot Linux From U-Boot

This is extremely useful during BSP development.

For example, conceptually:

```
fatload mmc 0:1 ${kernel_addr_r} zImage  
fatload mmc 0:1 ${fdt_addr_r} am335x-boneblack.dtb  
setenv bootargs 'console=tty00,115200n8 root=/dev/mmcblk0p2 rw rootwait'  
bootz ${kernel_addr_r} - ${fdt_addr_r}
```

The exact console device and memory addresses should match your U-Boot/kernel configuration.

This allows you to debug the boot sequence without changing the SD card repeatedly.

58. Fourth Debugging Point: Kernel → Rootfs

If Linux starts but displays:

```
VFS: Cannot open root device
```

or:

```
Kernel panic - not syncing:  
VFS: Unable to mount root fs
```

check:

```
root=/dev/mmcblk0p2
```

and verify that:

```
CONFIG_MMC  
CONFIG_EXT4_FS
```

are built into the kernel.

For root filesystem support, prefer:

```
CONFIG_EXT4_FS=y
```

rather than a module during initial boot.

59. Fifth Debugging Point: Rootfs → init

If the kernel mounts the rootfs but reports:

```
No init found
```

check:

```
/sbin/init
```

For BusyBox:

```
ls -l rootfs/sbin/init
```

It should point to BusyBox or be a valid executable.

For example:

```
/sbin/init -> /bin/busybox
```

Create the symlink if necessary:

```
ln -sf /bin/busybox rootfs/sbin/init
```

60. Sixth Debugging Point: C/C++ Application

If Linux boots but the application does not start:

Check:

```
ls -l /usr/bin/myapp
```

Check architecture:

```
file /usr/bin/myapp
```

Check libraries:

```
ldd /usr/bin/myapp
```

If `ldd` is available.

Typical error:


```
not found
```

means a required shared library is missing.

For example:

```
libstdc++.so.6 => not found
```

Copy the correct ARM library into:

```
/usr/lib
```

or:

```
/lib
```

inside the rootfs.

61. Application Debugging

First run manually:

```
/usr/bin/myapp
```

Do not initially start it automatically.

Confirm:

```
application starts  
application can access devices  
application can access network  
application can access filesystem
```

Only then add:

```
/usr/bin/myapp &
```

to the startup system.

62. Create a Better Application Service

Instead of directly starting the application from `rcS`, eventually create:

```
/etc/init.d/S99myapp
```

Example:

```
#!/bin/sh

case "$1" in
    start)
        echo "Starting myapp"
        /usr/bin/myapp &
        ;;

    stop)
        killall myapp
        ;;

    restart)
        killall myapp
        /usr/bin/myapp &
        ;;

    *)
        echo "Usage: $0 {start|stop|restart}"
        exit 1
        ;;
esac
```

Make executable:

```
chmod +x rootfs/etc/init.d/S99myapp
```

63. Add Logging

Create:

```
/var/log
```

Application can write:

```
/var/log/myapp.log
```

For example:

```
/var/log/  
├─ messages  
├─ kernel.log  
└─ myapp.log
```

For an embedded production system, logging strategy should be designed carefully because flash storage has finite write endurance.

64. Add Kernel Modules

If drivers are built as modules:

```
/lib/modules/<kernel-version>/
```

On the target:

```
uname -r
```

should match:

```
/lib/modules/<kernel-version>
```

For example:

```
/lib/modules/6.x.x/
```

Do not mix modules from another kernel build.

65. Add Firmware

If a driver requires firmware, place it in:

```
/lib/firmware/
```

Example:

```
rootfs/lib/firmware/
```

The kernel driver will request firmware from there.

66. Create an Application Development SDK

Once the BSP works, create a reusable SDK directory.

For example:

```
bbb-sdk/  
├─ bin/  
├─ sysroot/  
├─ include/  
├─ lib/  
├─ cmake/  
└─ environment-setup
```

Create an environment file:

```
export ARCH=arm  
export CROSS_COMPILE=/path/to/arm-linux-gnueabi-  
export SYSROOT=/path/to/bbb-sdk/sysroot  
  
export CC=${CROSS_COMPILE}gcc
```

```
export CXX=${CROSS_COMPILE}g++
export AR=${CROSS_COMPILE}ar
export LD=${CROSS_COMPILE}ld
export STRIP=${CROSS_COMPILE}strip

export CFLAGS="--sysroot=${SYSROOT}"
export CXXFLAGS="--sysroot=${SYSROOT}"
export LDFLAGS="--sysroot=${SYSROOT}"
```

Then:

```
source environment-setup
```

Now any application can use the same BSP environment.

67. CMake Application

Example:

```
myapp/
├─ CMakeLists.txt
├─ src/
│   └─ main.cpp
└─ include/
```

Toolchain file:

```
bbb-toolchain.cmake
```

Conceptually:

```
set(CMAKE_SYSTEM_NAME Linux)
set(CMAKE_SYSTEM_PROCESSOR arm)

set(CMAKE_C_COMPILER arm-linux-gnueabi-gcc)
set(CMAKE_CXX_COMPILER arm-linux-gnueabi-g++)

set(CMAKE_SYSROOT /path/to/bbb-sdk/sysroot)
```

Build:

```
cmake
  -DCMAKE_TOOLCHAIN_FILE=bbb-toolchain.cmake
  -S .
  -B build
```

Then:

```
cmake --build build
```

68. Add Libraries to BSP

If your application requires libraries such as:

```
OpenCV
SQLite
OpenSSL
Qt
custom libraries
```

you must cross-compile them for ARM and install them into the same sysroot.

For example:

```
sysroot/
├─ usr/include/
│   └─ sqlite3.h
├─ usr/lib/
│   ├── libsqlite3.so
│   └─ ...
└─ lib/
```

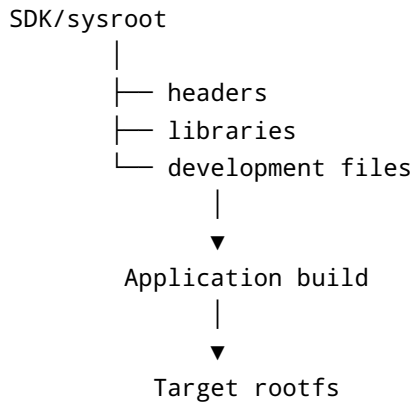
Then install the runtime libraries into:

```
rootfs/usr/lib/
```

or:

```
rootfs/lib/
```

This creates a clean separation:



69. GPIO Testing

After Linux boots:

```
ls /sys/class/gpio
```

On newer kernels, GPIO may instead be accessed through the GPIO character-device interface.

Check:

```
ls /dev/gpiochip*
```

This is preferable for modern applications.

The application should generally use a proper GPIO API rather than directly manipulating arbitrary kernel memory.

70. I2C Testing

Check:

```
ls /dev/i2c-*
```

You may see:

```
/dev/i2c-0  
/dev/i2c-1
```

Use an I2C utility package or your own C application to test devices.

Conceptually:

```
Application  
↓  
/dev/i2c-X  
↓  
Linux I2C driver  
↓  
AM335x I2C controller  
↓  
I2C pins  
↓  
External device
```

71. SPI Testing

Check:

```
ls /dev/spidev*
```

Example:

```
/dev/spidev0.0
```

Application:

```
C/C++  
↓  
spidev
```



```
↓  
SPI kernel driver  
↓  
AM335x SPI controller  
↓  
SPI device
```

72. UART Testing

Check:

```
ls /dev/tty*
```

Configure:

```
stty -F /dev/ttyS1 115200
```

Then your C application can open:

```
open("/dev/ttyS1", O_RDWR);
```

and communicate using:

```
read()  
write()  
ioctl()
```

73. Ethernet Testing

After boot:

```
ip link
```

Bring interface up:

```
ip link set eth0 up
```

Request DHCP if BusyBox provides `udhcpc` :

```
udhcpc -i eth0
```

Check:

```
ip addr
```

Then:

```
ping <gateway>
```

and:

```
ping 8.8.8.8
```

74. Create a Complete BSP Repository

Your project should eventually look like:

```
bbb-bsp/  
|  
├── README.md  
|  
├── toolchain/  
|  
├── sources/  
|   ├── u-boot/  
|   ├── linux/  
|   ├── busybox/  
|   ├── binutils/  
|   ├── gcc/  
|   └── glibc/  
|  
├── configs/  
|   └── uboot/
```

```
|
|   ├── kernel/
|   └── busybox/
|
|   ├── board/
|   │   └── beaglebone-black/
|   │       ├── dts/
|   │       ├── patches/
|   │       └── config/
|   │
|   ├── rootfs/
|   │   ├── bin/
|   │   ├── sbin/
|   │   ├── etc/
|   │   ├── dev/
|   │   ├── proc/
|   │   ├── sys/
|   │   ├── lib/
|   │   ├── usr/
|   │   └── var/
|   │
|   ├── application/
|   │
|   ├── scripts/
|   │   ├── build-toolchain.sh
|   │   ├── build-u-boot.sh
|   │   ├── build-kernel.sh
|   │   ├── build-rootfs.sh
|   │   ├── build-image.sh
|   │   └── flash-sd.sh
|   │
|   ├── images/
|   └── deploy/
```

75. Separate Source, Build and Deploy

A very important BSP development principle is:

```
Source
  ↓
Build
  ↓
Install
```

↓
Deploy

Do not mix everything into one directory.

For example:

```
sources/linux/
```

contains source.

```
build/linux/
```

contains build output.

```
deploy/kernel/
```

contains files that will actually be installed into the target image.

76. Create a BSP Build Script

A high-level build script should perform:

1. Build toolchain
2. Build U-Boot
3. Build Linux
4. Build device tree
5. Build kernel modules
6. Build BusyBox
7. Create rootfs
8. Install libraries
9. Build application
10. Install application
11. Create boot partition
12. Create rootfs partition
13. Generate final image

Conceptually:

```
./scripts/build.sh
```

should produce:

```
images/  
└─ beaglebone-black.img
```

77. Create Complete Flash Image

There are two approaches.

Approach A — Flash SD partitions directly

This is easiest during development:

```
SD card  
├─ FAT32 boot  
└─ EXT4 rootfs
```

Approach B — Generate one `.img`

For production, generate:

```
beaglebone-black.img
```

containing:

```
MBR  
├─  
├─ Boot partition  
│   ├── MLO  
│   ├── u-boot.img  
│   ├── zImage  
│   └─ DTB  
└─ Rootfs partition  
    ├── /bin  
    └─ /etc
```

```
|— /lib
|— /usr
└─ application
```

Then the complete image can be written with:

```
sudo dd if=beaglebone-black.img
      of=/dev/sdX
      bs=4M
      status=progress
      conv=fsync
```

Be extremely careful with `/dev/sdX`.

Using the wrong device can destroy your host PC's disk.

78. Recommended Image Layout

A practical BBB image:

```
beaglebone-black.img
|
|— MBR
|
|— Partition 1
|   FAT32
|   128-256 MB
|
|   |— MLO
|   |— u-boot.img
|   |— zImage
|   |— am335x-boneblack.dtb
|   └─ extlinux/
|       └─ extlinux.conf
|
└─ Partition 2
    EXT4
    Remaining space
    |
    |— bin/
    |— sbin/
    └─ etc/
```

```
├── dev/
├── lib/
├── proc/
├── sys/
├── usr/
│   ├── bin/
│   └── myapp
└── var/
```

79. eMMC Boot

Once SD boot works, move to eMMC.

Do **not** start development by repeatedly flashing eMMC.

First establish:

```
graph TD; SD_boot[SD boot] --> U_Boot[U-Boot]; U_Boot --> Kernel[Kernel]; Kernel --> Rootfs[Rootfs]; Rootfs --> Application[Application];
```

Then migrate to eMMC.

The eMMC is normally visible as an MMC block device.

Check:

lsblk

You may see devices such as:

```
mmcb1k0
mmcb1k1
```

The exact numbering depends on the boot configuration and kernel.

80. Flashing eMMC

The safest method during development is generally:

```
Boot BBB from SD
  ↓
Linux running from SD
  ↓
Identify eMMC
  ↓
Write boot/rootfs image to eMMC
  ↓
Power off
  ↓
Remove SD
  ↓
Boot from eMMC
```

Before writing anything:

```
lsblk
```

Identify which device is eMMC.

Never blindly assume:

```
/dev/mmcblk1
```

or:

```
/dev/mmcblk0
```

is the eMMC.

Verify from:


```
cat /proc/partitions
```

and:

```
ls -l /dev/mmc*
```

81. Boot Source Selection

The AM335x supports different boot sources.

Conceptually:

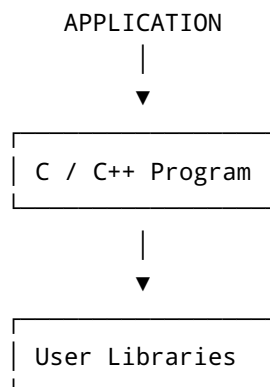
```
Boot pins
|
├── eMMC
├── SD
├── UART
└── USB
```

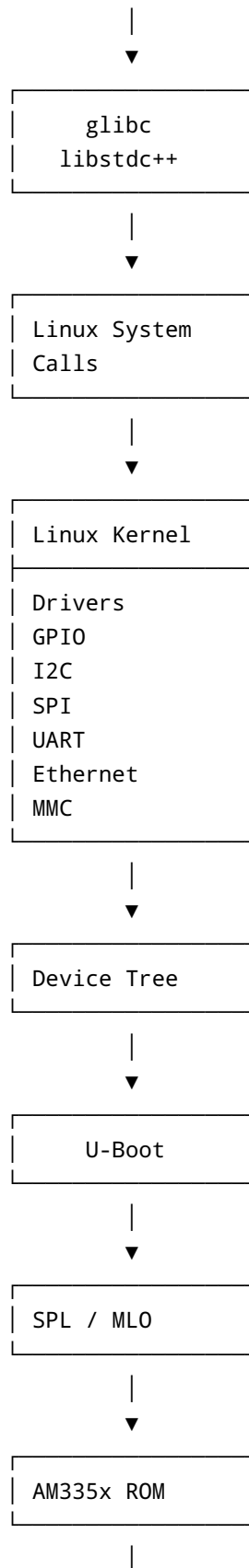
For development, SD boot is highly convenient because the original eMMC remains untouched.

The BBB boot configuration determines which media the ROM attempts to boot from.

82. Full BSP Architecture

The complete software architecture should look like:





83. What You Have Built

At the end of the process, you have created:

1. Cross compiler

```
arm-linux-gnueabihf-gcc  
arm-linux-gnueabihf-g++
```

2. Bootloader

```
SPL  
U-Boot
```

3. Kernel

```
zImage
```

4. Device Tree

```
am335x-boneblack.dtb
```

5. Root filesystem

```
BusyBox  
glibc  
libgcc  
libstdc++  
configuration  
device filesystem
```

6. Kernel modules

```
/lib/modules/
```

7. Application

```
/usr/bin/myapp
```

8. Boot configuration

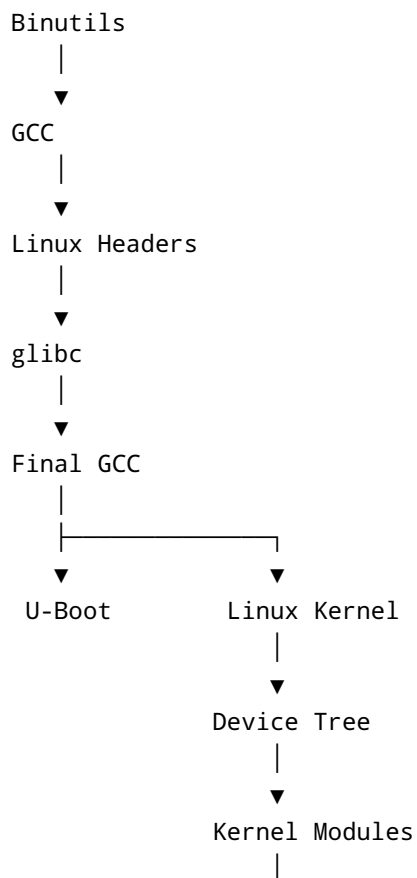
```
extlinux.conf
```

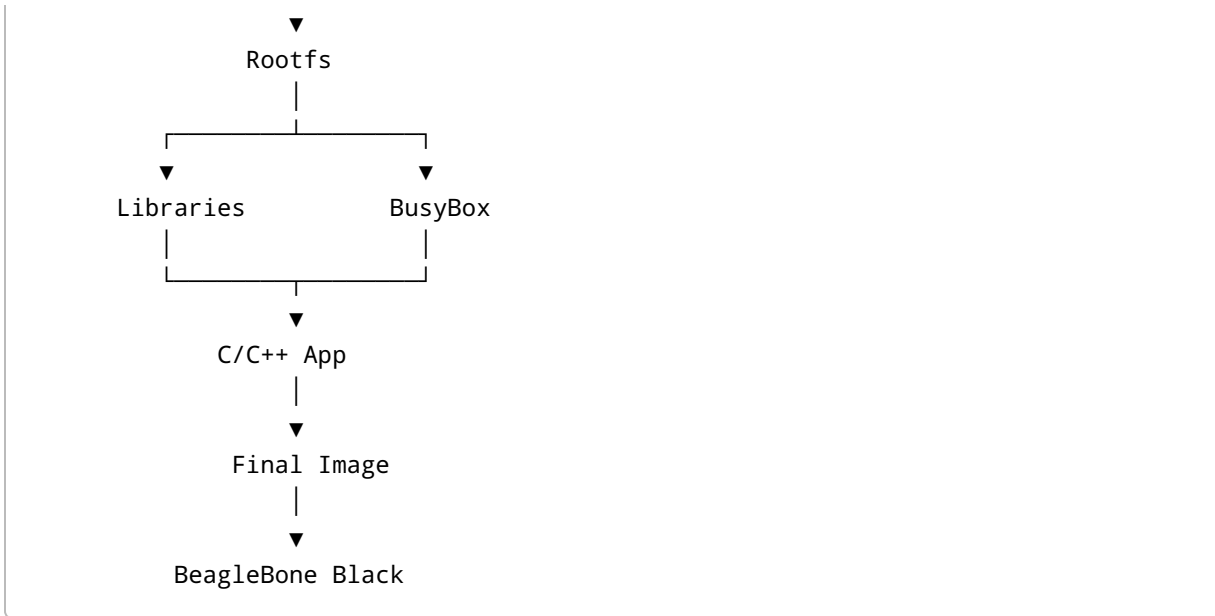
9. Flash image

```
beaglebone-black.img
```

84. End-to-End Build Dependency

The important dependency chain is:





85. Recommended Development Sequence

Do not attempt to build everything simultaneously.

Follow this sequence.

Phase 1 — Toolchain

- [] Install host development packages
- [] Prepare BSP directory
- [] Prepare ARM cross compiler
- [] Verify ARM compiler
- [] Prepare sysroot

Phase 2 — Bootloader

- [] Obtain U-Boot source
- [] Configure AM335x
- [] Build SPL
- [] Build U-Boot
- [] Verify MLO
- [] Verify u-boot.img

Phase 3 — Kernel

```
[ ] Obtain Linux source
[ ] Configure ARM
[ ] Enable AM335x
[ ] Enable MMC
[ ] Enable EXT4
[ ] Enable UART
[ ] Enable Ethernet
[ ] Enable GPIO
[ ] Enable I2C
[ ] Enable SPI
[ ] Build zImage
[ ] Build DTB
[ ] Build modules
```

Phase 4 — Rootfs

```
[ ] Build BusyBox
[ ] Create rootfs directories
[ ] Create passwd
[ ] Create group
[ ] Create hostname
[ ] Create inittab
[ ] Create rcS
[ ] Mount proc
[ ] Mount sysfs
[ ] Mount devtmpfs
```

Phase 5 — Libraries

```
[ ] Install glibc
[ ] Install libgcc
[ ] Install libstdc++
[ ] Install required shared libraries
[ ] Verify dynamic loader
```

Phase 6 — Application

```
[ ] Create C application
[ ] Cross compile
[ ] Verify ARM binary
```

```
[ ] Install into /usr/bin
[ ] Run manually
[ ] Add startup service
```

Phase 7 — SD Card

```
[ ] Create FAT partition
[ ] Create EXT4 partition
[ ] Install MLO
[ ] Install U-Boot
[ ] Install kernel
[ ] Install DTB
[ ] Install extlinux.conf
[ ] Install rootfs
```

Phase 8 — Boot

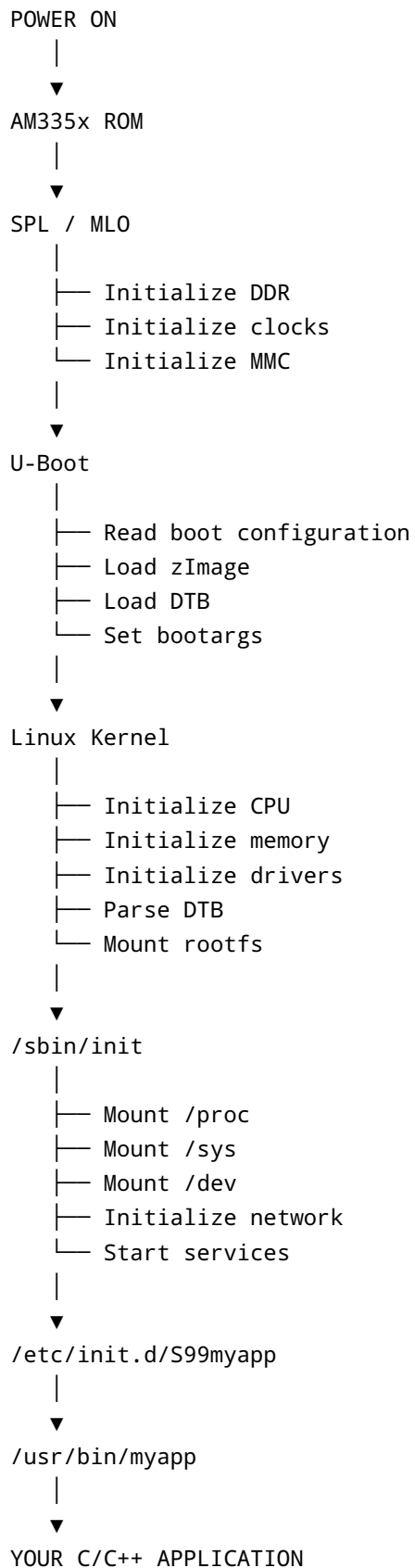
```
[ ] Insert SD
[ ] Connect UART
[ ] Power board
[ ] Verify SPL
[ ] Verify U-Boot
[ ] Verify kernel
[ ] Verify rootfs
[ ] Verify init
[ ] Verify application
```

Phase 9 — eMMC

```
[ ] Boot from SD
[ ] Identify eMMC
[ ] Flash image
[ ] Configure boot source
[ ] Remove SD
[ ] Boot from eMMC
```

86. Final Boot Sequence

The finished system should operate like this:



87. Important Difference Between BSP and OS

It is important to use the terminology correctly.

A BSP is **not only the kernel**.

For this BBB project:

```
BSP =  
  Bootloader  
+ Kernel  
+ Device Tree  
+ Board Drivers  
+ Firmware  
+ Toolchain/Sysroot  
+ Board Configuration
```

The complete embedded Linux system is:

```
Complete OS =  
  BSP  
+ Root Filesystem  
+ Libraries  
+ Init System  
+ Services  
+ Application
```

Therefore, when someone asks you to create a complete BBB Linux platform from scratch, the engineering deliverables should be:

1. Toolchain
2. U-Boot
3. SPL
4. Linux kernel
5. Device Tree
6. Kernel modules
7. Root filesystem
8. C/C++ runtime
9. Application
10. Boot configuration
11. SD/eMMC image
12. Flashing script

- 13. BSP build scripts
- 14. BSP documentation

88. Suggested Final Project Structure

A production-oriented project can finally look like:

```
BeagleBoneBlack-BSP/
|
├── README.md
|
├── toolchain/
|
├── sysroot/
|
├── sources/
|   ├── u-boot/
|   ├── linux/
|   ├── busybox/
|   ├── gcc/
|   ├── glibc/
|   └── binutils/
|
├── board/
|   └── beaglebone-black/
|       ├── dts/
|       ├── patches/
|       ├── kernel.config
|       ├── uboot.config
|       └── busybox.config
|
├── rootfs/
|   ├── bin/
|   ├── sbin/
|   ├── etc/
|   ├── lib/
|   ├── usr/
|   ├── var/
|   ├── dev/
|   ├── proc/
|   └── sys/
|
└── application/
```

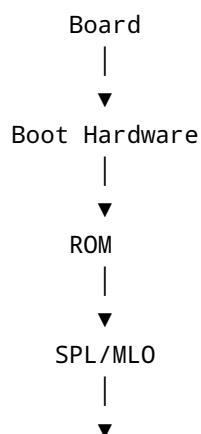
```
|   └─ myapp/
|
|   └─ scripts/
|       ├── setup-toolchain.sh
|       ├── build-toolchain.sh
|       ├── build-uboot.sh
|       ├── build-kernel.sh
|       ├── build-busybox.sh
|       ├── build-rootfs.sh
|       ├── build-app.sh
|       ├── create-image.sh
|       └─ flash-emmc.sh
|
|   └─ configs/
|       ├── kernel/
|       ├── uboot/
|       └─ busybox/
|
|   └─ images/
|
|   └─ deploy/
```

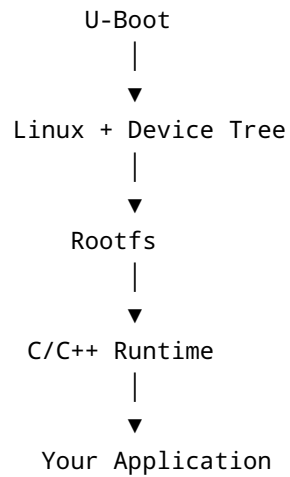
89. The Most Important Concept

If your goal is to learn **real BSP development**, do not think of the task as:

"Build Linux"

Think of it as:





Every layer must be independently verified.

The recommended rule is:

Do not move to the next layer
until the current layer is proven to work.

For example:

```
SPL works
↓
U-Boot works
↓
Kernel works
↓
DTB works
↓
Rootfs works
↓
Shell works
↓
Libraries work
↓
Application works
↓
SD image works
↓
eMMC image works
```

That is the fundamental end-to-end approach for creating a **BeagleBone Black BSP and complete embedded Linux OS without Yocto/Buildroot.**